

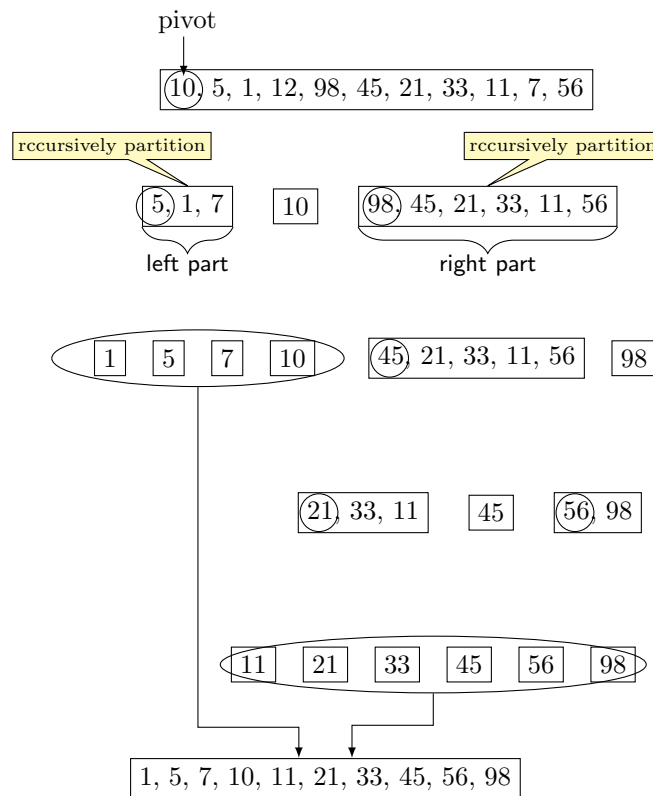
Quick sort algorithm

QuickSort is an efficient method for partitioning a given set of elements into two subsets by utilizing one of the elements as the pivot. This pivot is positioned between the two subsets, where the first subset contains elements that are less than or equal to the pivot, while the second subset includes elements greater than the pivot. The pivot is often referred to as the partitioning element. Once the subsets are formed, we recursively apply the partitioning process to each subset. This procedure continues until each subset is reduced to a single element.

Partitioning is a crucial operation in the quick sort algorithm. It is essential to select the partitioning element thoughtfully to ensure the given set of elements is divided into two subsets of approximately equal size. If the partitioning is unbalanced, the sorting process may degrade into an inefficient method, resulting in a running time of $O(n^2)$. For instance, one partition might consist of $O(n)$ elements while the other contains only $O(1)$ elements.

How do we select the partitioning element? Intuitively, the median of the given set would be the optimal choice. However, determining the median can be computationally as demanding as sorting the entire set. There is a more efficient $O(n)$ method to find the median, though explaining this approach can be quite intricate. Given that the input set is random, using the first element as the partitioning element often results in a balanced partitioning of the elements. Alternatively, simpler techniques can be employed for selecting a partitioning element. For example, one could randomly choose three elements and select the median of those three.

Figure below illustrates an example of executing quick sort algorithm,



Initially, the number 10 is placed at the beginning of the array, serving as the pivot for the sorting process.

The array is partitioned into two distinct subarrays based on this pivot:

1. The left subarray: [5, 1, 7]
2. The right subarray: [98, 45, 21, 33, 11, 56]

Next, we apply recursive partitioning to these two subarrays while keeping the pivot 10 in the first position, as illustrated in the accompanying figure. The pivots chosen for further partitioning are 5 for the left subarray and 98 for the right subarray. The circle around the elements in the figure above, identifies the pivots we have used for different partitions.

The left subarray undergoes another partitioning step, which results in a fully sorted sequence: [1, 5, 7, 10]. In contrast, the partitioning of the right subarray with pivot 98 leads to imbalanced partitions:

1. The left subarray: [45, 21, 33, 11, 56]

2. The right subarray: []

In the next stage of partitioning, we select 45 as the pivot for the left subarray. Eventually, after another round of partitioning, we achieve the sorted left subarray: [11, 21, 33, 45, 56, 98].

To arrive at the final sorted array, we concatenate the sorted left and right subarrays, resulting in the complete sorted sequence: [1, 5, 7, 10, 11, 21, 33, 45, 56, 98]. This example effectively demonstrates the recursive nature of the quicksort algorithm and its ability to efficiently sort an array.

The key aspect of the proof of correctness is that the partitioning process places the pivot in its correct index position within the array. When the array contains a single element, the partition operation does not alter it. To prove correctness of quick sort for an array of size n , we assume that it operates correctly for an array of size k and proceed to demonstrate that it will also work correctly for an array of size $k + 1$.

After applying the partition operation to an array of size $k + 1$, we obtain three partitions:

- One partition containing elements that are less than the pivot.
- One partition containing the pivot itself.
- One partition containing elements that are greater than the pivot.

Since the partition containing the pivot has a cardinality of one, the cardinalities of the other two partitions must each be less than $k + 1$. Hence, the induction hypothesis is applicable to both of these partitions. Additionally, as the position of the pivot is correctly determined, quicksort successfully rearranges the elements of the array into sorted order.